電影管理 新增・修改・上下架 流程圖

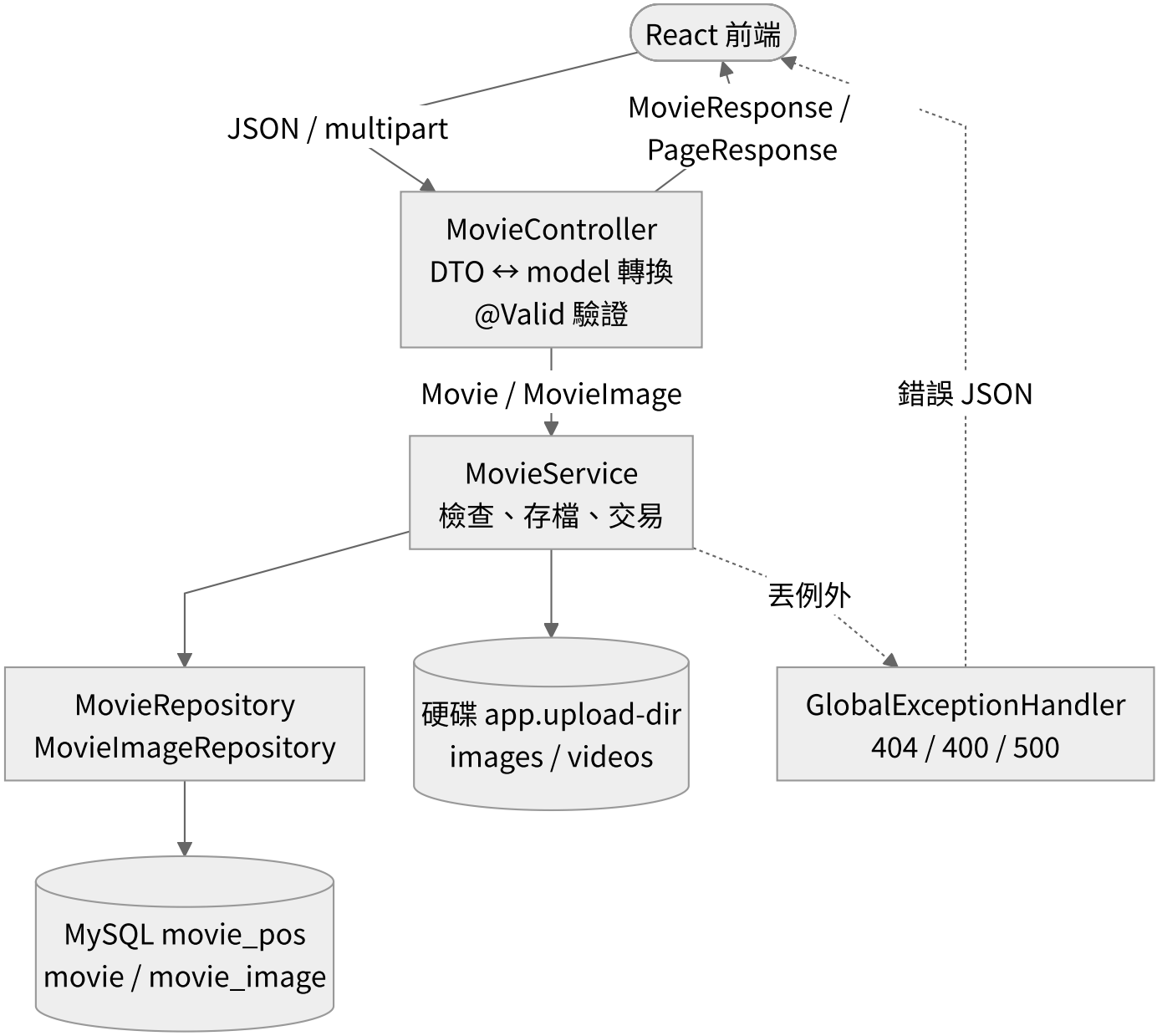
movie 專題（Spring Boot + MySQL movie_pos）— Movie / MovieImage

功能	API	Service 方法	成功
查詢上架（分頁）	`GET /api/movie?page=1&size=10`	`findIsActive`	200
查詢單筆	`GET /api/movie/{id}`	`findMovieByMovieId`	200
新增	`POST /api/movie/add`	`createMovie`	201
修改	`PUT /api/movie/{id}`	`updateMovie`	200
查詢下架（分頁）	`GET /api/movie/inactive?page=1&size=10`	`findIsInactive`	200
上架 / 下架	`PUT /api/movie/{id}`（isActive: true / false）	`updateMovie`	200

錯誤回應：找不到電影 → 404；資料有誤、@Valid 失敗、缺少 movie、JSON 格式錯誤、id 不是數字 → 400；Content-Type 沒設 application/json → 415；檔案太大 → 413；檔案寫入失敗 → 500。
電影不提供刪除：is_active = 1 上架、0 下架，資料與檔案一律保留。
依 2026-10-31 專案程式碼繪製。

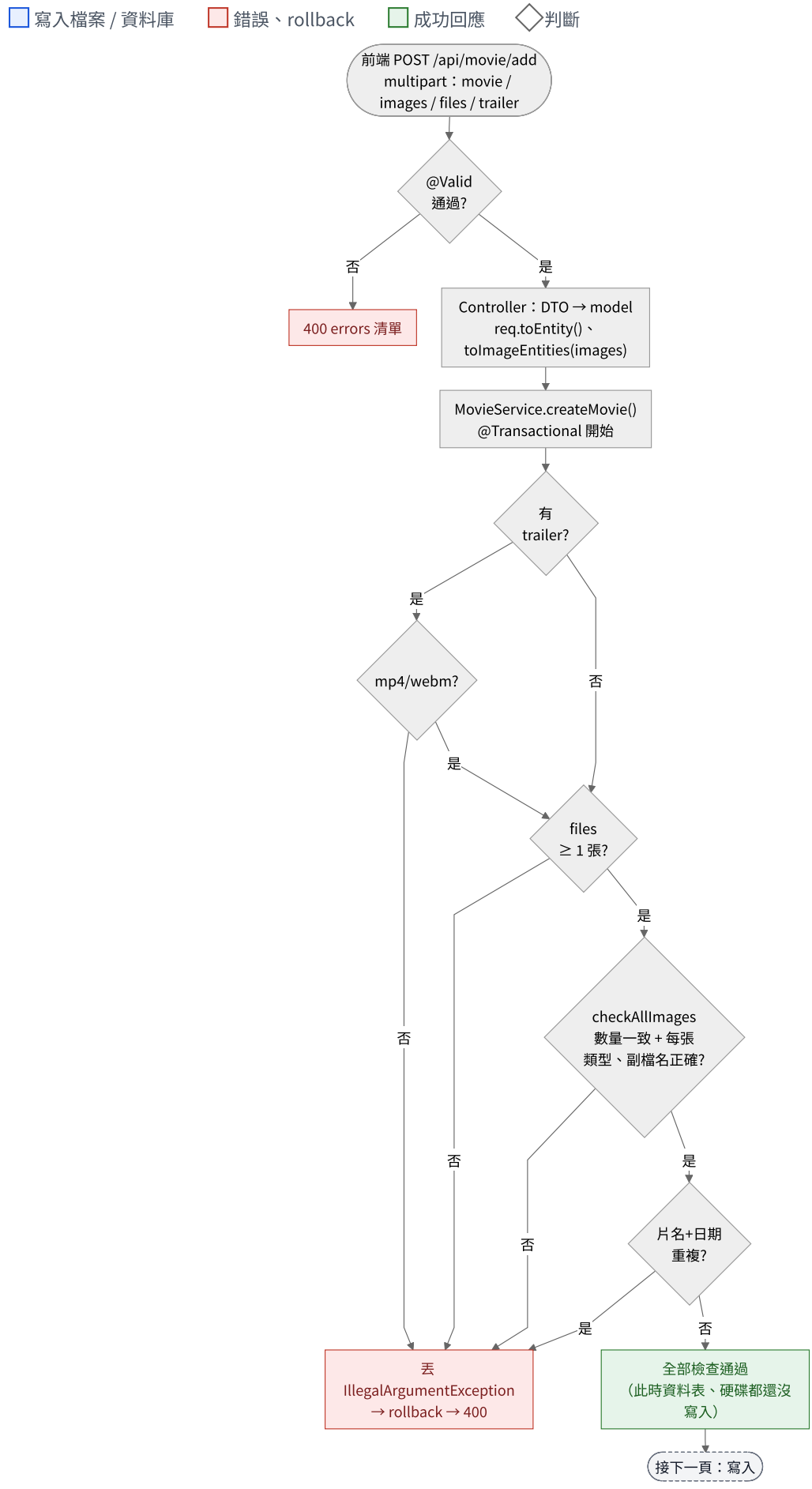
1 整體架構

Controller 只負責 DTO 與 model 互轉；Service 只收、只回傳 model（Movie / MovieImage），失敗一律丟例外，由 GlobalExceptionHandler 統一轉成錯誤回應。



❷ 新增電影（1/2）檢查階段　POST /api/movie/add

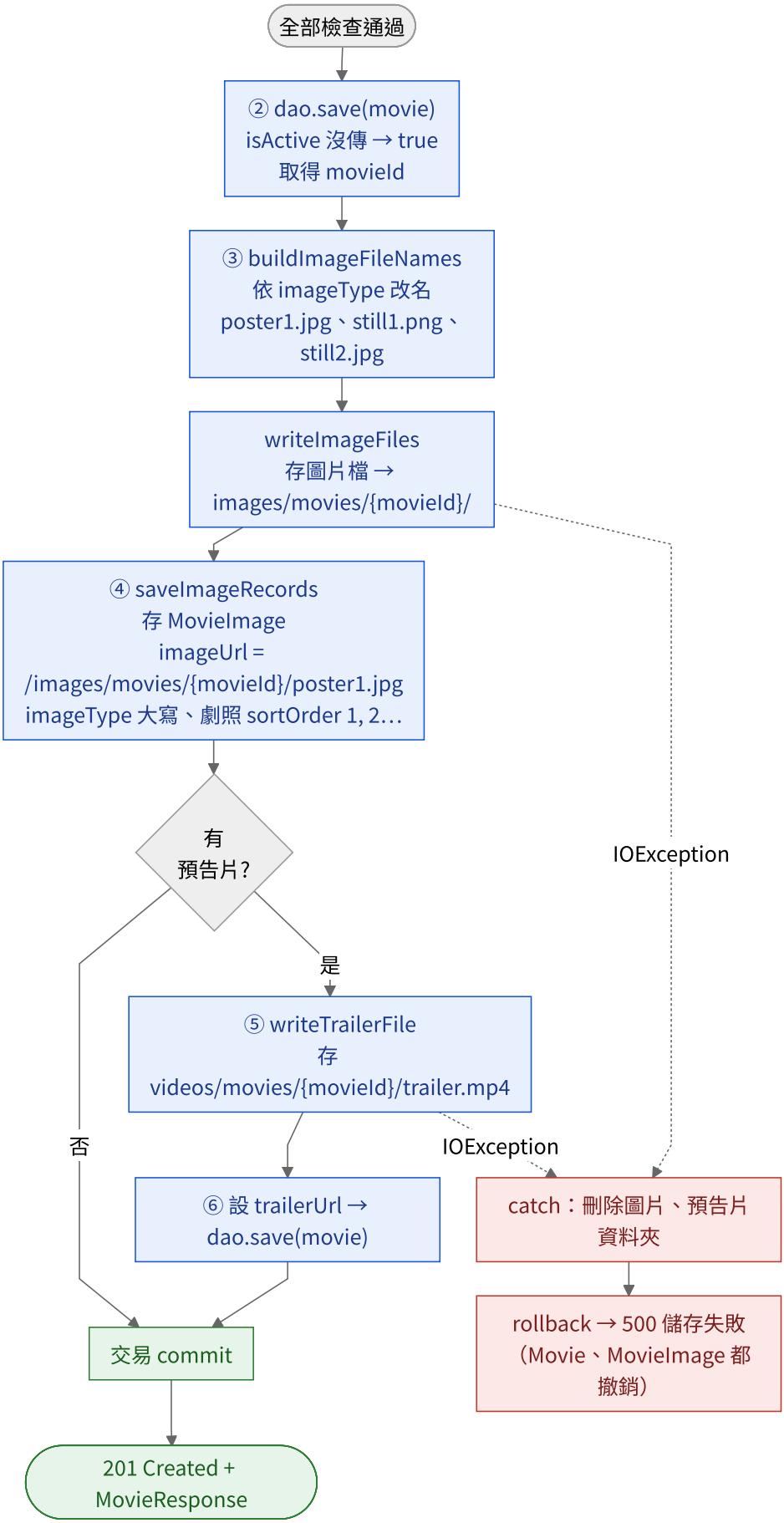
寫入任何東西之前先「全部檢查」：預告片副檔名、圖片數量、每張圖的類型與副檔名、片名 + 上映日期是否重複。**只要有一項沒過，就不會存資料表，也不會存任何檔案。**



❸ 新增電影（2/2）寫入階段

檢查通過後才寫入：存 Movie 取得 movieId → 依 imageType 改名並存圖片檔 → 存 MovieImage（相對路徑）→ 存預告片檔 → 更新 trailerUrl。新增時不會刪除任何舊路徑；寫檔失敗會刪掉資料夾並 rollback。

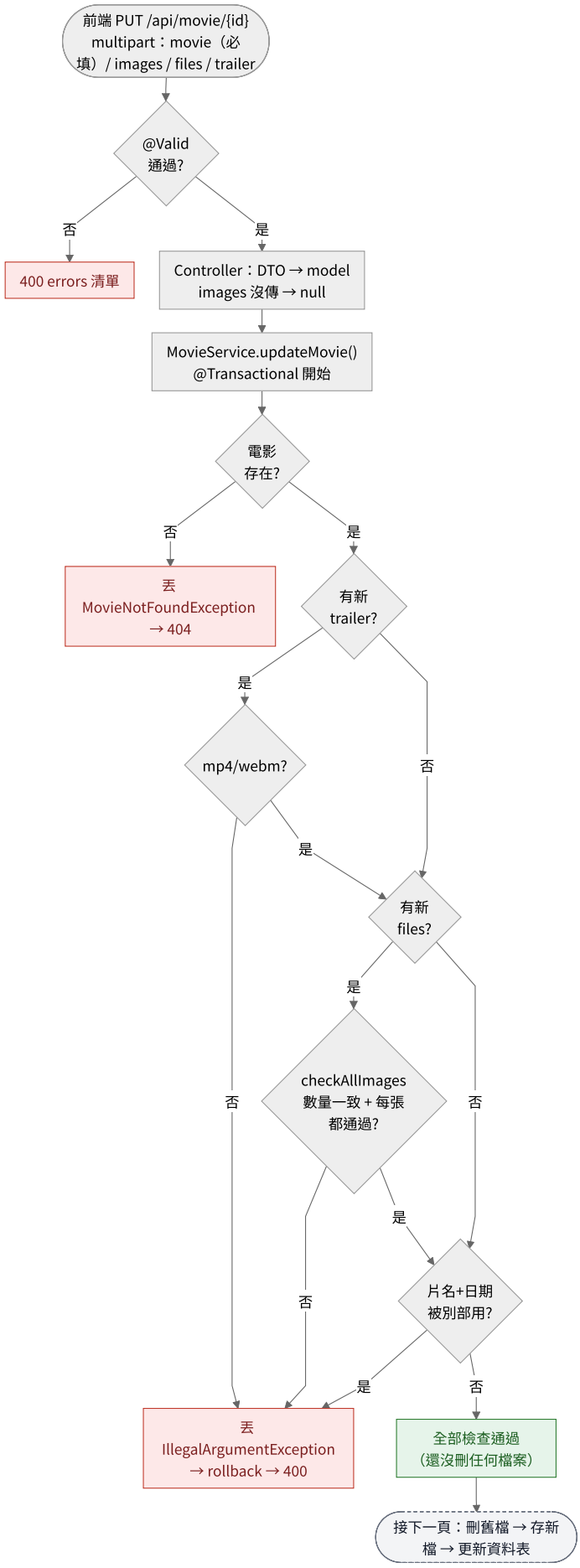
寫入檔案 / 資料庫　錯誤、rollback　成功回應　判斷



❹ 修改電影（1/2）檢查階段　PUT /api/movie/{id}

沒附圖檔、預告片檔就不檢查、不更動。全部檢查通過之前，不會刪除任何原本的檔案。

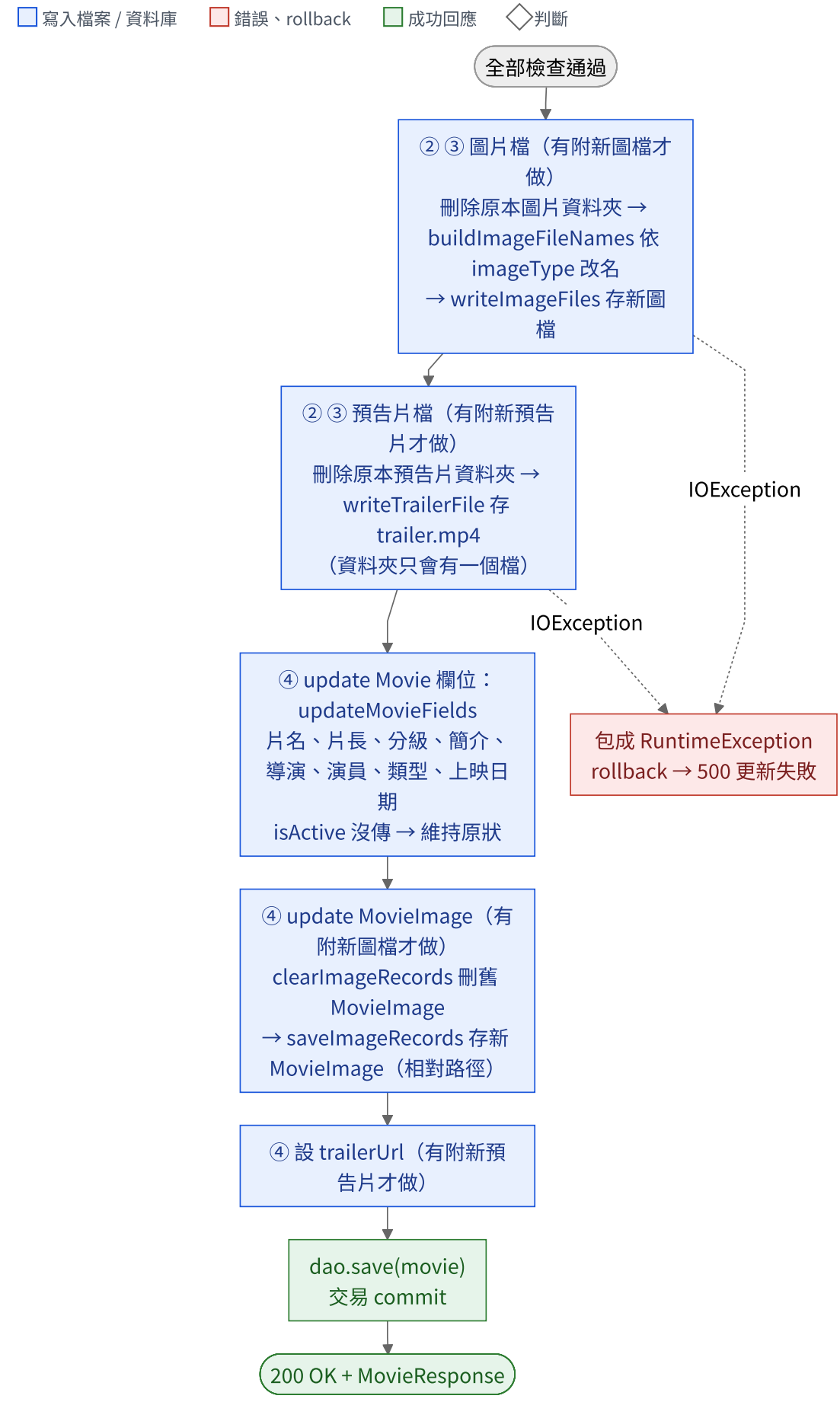
寫入檔案 / 資料庫　錯誤、rollback　成功回應　判斷



5 修改電影（2/2）寫入階段

順序：刪除原本圖檔 / 預告片檔 → 存新檔 → update Movie 欄位 → 刪舊 MovieImage、存新 MovieImage → 更新 trailerUrl → commit。

⚠ 舊檔案在更新資料表之前就刪掉，若資料表那步失敗會 rollback，但舊檔案無法復原。



6 上架 / 下架與查詢

電影不會被刪除。movie、movie_image 的資料一律保留，只用 is_active 區分：1 = 上架、0 = 下架。下架就是用修改 API 把 isActive 設成 false；前台只查上架電影，後台可另外查下架電影。

寫入檔案 / 資料庫　錯誤、rollback　成功回應　判斷

